



École Nationale Supérieure d'Informatique
pour l'Industrie et l'Entreprise

Rapport de stage de première année

Conception d'un tableau de bord interne

ÉTUDIANT

Omar DAHMAN

TUTEUR ACADÉMIQUE

David ROUSSEL

MAÎTRE DE STAGE

Louail POTHIN

ENTREPRISE

RT MOVE



RT MOVE — 26 Rue de la Frette, 78500 Sartrouville

1^{er} juillet — 28 août 2026

Remerciements

Je tiens à remercier mon maître de stage, Louail Pothin, pour son accueil, sa disponibilité et les conseils qu'il m'a donnés tout au long de ce stage. Je remercie également le gérant de RT MOVE de m'avoir permis de réaliser ce stage au sein de son entreprise et de m'avoir fait confiance sur ce projet. Merci aussi à mon tuteur académique, David Roussel, pour son suivi et son accompagnement pendant ces deux mois. Enfin, je remercie Aurélien, Mounib, Israa et David, les quatre autres stagiaires, pour la bonne entente et l'entraide qui ont rendu ce travail d'équipe agréable et efficace.

Résumé

Ce stage, réalisé chez RT MOVE (entreprise de déménagement) du 1^{er} juillet au 28 août 2026, s'inscrit dans un projet de digitalisation mené par une équipe de cinq stagiaires. Ma mission (lot L5) a consisté à concevoir un **tableau de bord web interne de pilotage en temps réel** : cartographie des camions et des missions, calcul d'itinéraires, suivi temps réel des positions, indicateurs commerciaux et marketing, et alertes opérationnelles.

Le tableau de bord a été développé en Next.js/React/TypeScript, avec Leaflet et OpenRouteService pour la cartographie, et Supabase (Realtime et *Row Level Security*) pour les données. Une attention particulière a été portée à la **robustesse face aux dépendances inter-équipes** : chaque flux temps réel bascule automatiquement en interrogation périodique (*polling*) si le canal Realtime est indisponible, et l'intégration à la base partagée a été validée par des tests directs de l'API.

[A COMPLETER : Compléter avec les résultats finaux (démonstration au gérant, recette) une fois la fin du stage atteinte.]

Glossaire

Next.js / React Cadriciels JavaScript/TypeScript pour construire des interfaces web.

Supabase Plateforme fondée sur PostgreSQL fournissant base de données, authentification, stockage et abonnements temps réel.

RLS (Row Level Security) Mécanisme PostgreSQL qui restreint l'accès aux lignes d'une table selon le rôle/l'utilisateur connecté.

Realtime Canal d'abonnement de Supabase qui pousse en direct les changements de la base (`postgres_changes`) vers le client, sans rechargement.

Polling Interrogation périodique de la base, utilisée ici en repli quand Realtime est indisponible.

Leaflet / OpenStreetMap (OSM) Bibliothèque de cartographie et fond de carte libre.

OpenRouteService (ORS) Service de calcul d'itinéraires et de temps de trajet estimé (ETA).

KPI *Key Performance Indicator*, indicateur clé de performance (commercial, marketing).

ADR *Architecture Decision Record*, note documentant un choix technique.

PR (Pull Request) Demande de fusion d'une branche Git vers la branche principale, soumise à relecture avant intégration.

Jalon (J1–J6) Point de validation commun à l'équipe, aux dates clés du stage.

Table des figures

3.1 Les quatre pages du tableau de bord 4

Table des matières

Remerciements	i
Résumé	ii
Glossaire	iii
Table des figures	iv
1 Introduction : environnement de travail et stage	1
1.1 L'entreprise RT MOVE	1
1.2 Contexte du projet de digitalisation	1
1.3 Une équipe de cinq stagiaires	1
1.4 Ma mission (lot L5) et ses objectifs	2
2 Contexte et préliminaires	3
2.1 Pile technique	3
2.2 Notions clés	3
2.2.1 Cartographie web et itinéraires	3
2.2.2 Temps réel : Realtime contre polling	3
2.2.3 Sécurité des données (RLS)	3
2.3 Mise en place de l'environnement	3
3 Solution implémentée et contributions	4
3.1 Architecture générale	4
3.2 Cartographie et itinéraires	5
3.3 Suivi temps réel et robustesse	5
3.4 Alertes de pilotage	5
3.5 Indicateurs métier (KPI)	5
3.6 Intégration à la base Supabase	5
3.6.1 Principe et politique de sécurité	5
3.6.2 Un blocage diagnostiqué puis résolu	5
3.7 Intégration avec le socle du CRM	6
3.8 Refonte du design	6
3.9 Démarche de vérification	6
3.10 Difficultés rencontrées	6
3.10.1 Un cache de compilation corrompu	6
3.10.2 Des droits de base non ouverts au départ	6
3.10.3 Dépendances inter-équipes	7

4 Conclusion et prochaines étapes	8
4.1 État d'avancement au 11 août 2026	8
4.2 Prochaines étapes	8
A DD&RS — Développement durable et responsabilité sociétale	9
B Structure de l'entreprise	10
C Détails techniques	11
C.1 Hook de suivi temps réel avec repli polling	11
C.2 Alertes de pilotage	11
C.3 Appel sécurisé à OpenRouteService (côté serveur)	11
Bibliographie et webographie	12

Chapitre 1

Introduction : environnement de travail et stage

1.1 L'entreprise RT MOVE

RT MOVE est une entreprise spécialisée dans le transport, le déménagement et le nettoyage en Île-de-France. Dans le cadre de son développement, l'entreprise souhaite améliorer ses outils numériques afin de mieux gérer ses opérations, centraliser ses données et automatiser son acquisition de clients professionnels : c'est l'objet du projet de digitalisation décrit dans ce rapport.

L'entreprise est dirigée par deux personnes, enregistrées au Registre National des Entreprises (RNE) tenu par l'INPI :

- **Louail Giovanni POTHIN**, directeur général — également mon maître de stage ;
- **Thomas ROLLOT**, président de SAS.

1.2 Contexte du projet de digitalisation

Avant ce projet, l'activité commerciale et opérationnelle de RT MOVE était pilotée à partir de classeurs Excel (prospection, closing, suivi des missions, contenus marketing). Ce fonctionnement montrait ses limites : saisie manuelle, absence de vision temps réel, difficulté à consolider les indicateurs. Le projet vise à remplacer progressivement ces classeurs par un système intégré (base de données, CRM, application mobile terrain, automatisations et tableau de bord).

1.3 Une équipe de cinq stagiaires

Le projet est découpé en lots (L0 à L8) répartis entre cinq stagiaires :

- **David** — Fondations & DevOps, automatisation prospection/marketing (L0, L6).
- **Aurélien** — Product Owner & BI, modélisation des données, front-end CRM, recette (L1, L3, L8).
- **Mounib** — Backend & cybersécurité, base de données, API, sécurité (L1, L2, L4).
- **Israa** — Application mobile terrain, suivi GPS (L7).
- **Omar (moi)** — Tableau de bord web temps réel (L5).

1.4 Ma mission (lot L5) et ses objectifs

Mon lot consiste à concevoir le tableau de bord interne de pilotage :

1. une **cartographie** des camions et des missions du jour (Leaflet + OSM), avec itinéraires et temps de trajet estimés (OpenRouteService) ;
2. un **suivi temps réel** des positions GPS remontées par l'application mobile ;
3. des **indicateurs métier** (entonnoir commercial, chiffres marketing) ;
4. une vue de **pilotage du jour** avec alertes (lead chaud, mission en retard).

Chapitre 2

Contexte et préliminaires

2.1 Pile technique

Le tableau de bord s'appuie sur : Next.js 16, React 19, TypeScript, Tailwind CSS 4 et shadcn/ui pour l'interface ; Leaflet et OpenStreetMap pour la carte ; OpenRouteService pour les itinéraires ; Recharts pour les graphiques ; et le client JavaScript Supabase (Auth, Realtime, RLS) pour les données. Le tout est hébergé dans le même dépôt mono-repo que le CRM, dont je réutilise le socle applicatif partagé.

2.2 Notions clés

2.2.1 Cartographie web et itinéraires

Leaflet affiche un fond de carte OpenStreetMap et des marqueurs personnalisés (camions, points de départ/arrivée des missions). OpenRouteService calcule, à partir de coordonnées, un itinéraire routier (polyligne) et un temps de trajet estimé (ETA).

2.2.2 Temps réel : Realtime contre polling

Deux approches permettent d'actualiser la carte sans rechargement : un canal *push* (Supabase Realtime, qui notifie les changements de la base) ou une interrogation périodique (*polling*). Le choix, documenté dans un ADR (annexe C), retient Supabase Realtime comme canal principal, avec repli automatique en polling.

2.2.3 Sécurité des données (RLS)

Les tables sont protégées par des politiques *Row Level Security* : seules les requêtes authentifiées (rôle `authenticated`) peuvent lire les données. Le tableau de bord doit donc être connecté pour afficher la moindre ligne (voir section 3.6).

2.3 Mise en place de l'environnement

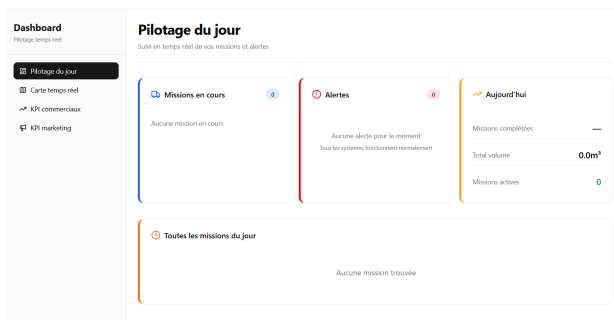
L'environnement de développement (dépôt, conventions Git, projet Supabase, intégration continue) a été fourni par David (lot L0). Après clonage, l'installation locale et le lancement du serveur de développement se font en quelques minutes grâce au guide d'onboarding partagé.

Chapitre 3

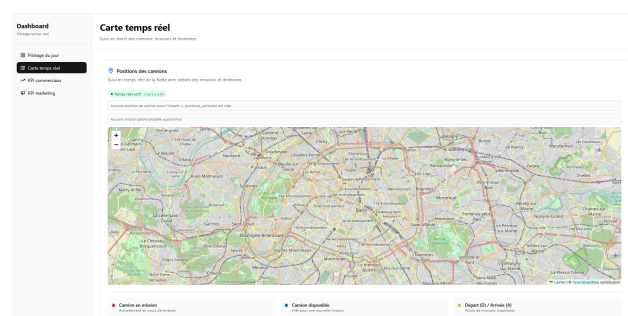
Solution implémentée et contributions

3.1 Architecture générale

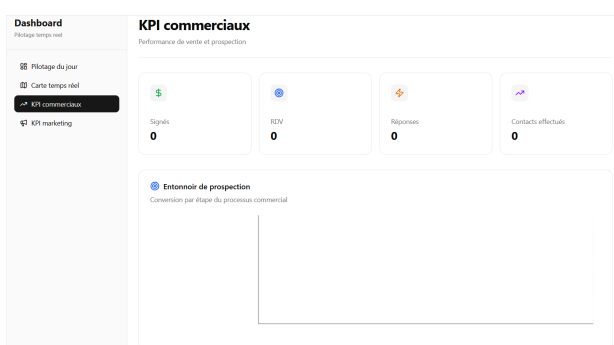
Le tableau de bord est organisé en quatre pages sous `/admin/dashboard` : *pilotage du jour*, *carte temps réel*, *KPI commerciaux* et *KPI marketing*. Il s'insère sous le socle applicatif partagé du CRM (navigation, garde d'authentification) construit par Aurélien, via un *layout imbriqué* qui ajoute la sous-navigation propre au dashboard.



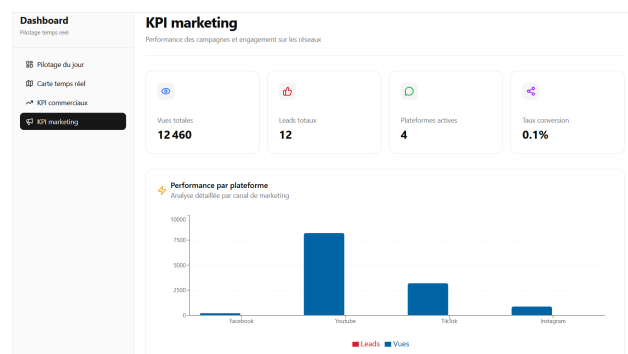
(a) Pilotage du jour



(b) Carte temps réel



(c) KPI commerciaux



(d) KPI marketing

FIGURE 3.1 – Les quatre pages du tableau de bord

3.2 Cartographie et itinéraires

La carte affiche les camions et les missions du jour. Les adresses sont géocodées, puis OpenRouteService calcule l'itinéraire et l'ETA. Les appels à ORS passent par une route serveur Next.js qui protège la clé d'API et gère les erreurs (quota dépassé : code 429, service indisponible : code 502).

3.3 Suivi temps réel et robustesse

Les positions des véhicules sont lues dans la table `positions_vehicules` et mises à jour en direct via un abonnement Realtime. Un point clé de la conception est la **robustesse** : si le canal Realtime n'est pas disponible (réplication non activée, coupure réseau), l'interface bascule automatiquement en polling, et un badge indique l'état de la connexion (« Temps réel actif » / « Mode dégradé ») ainsi que la fraîcheur de la dernière mise à jour.

3.4 Alertes de pilotage

La vue de pilotage remonte deux types d'alertes sur données réelles :

- **Lead chaud** — tout nouveau prospect inséré dans la table `prospects`. Le moteur de scoring de David ne remonte que les prospects au-dessus du seuil ; un nouveau prospect équivaut donc à un lead à signaler. L'abonnement Realtime sur les INSERT affiche l'alerte immédiatement.
- **Mission en retard** — mission dont la date d'opération est passée alors que son statut n'est ni « terminée » ni « annulée ».

3.5 Indicateurs métier (KPI)

Trois familles d'indicateurs sont proposées : un *entonnoir commercial* (contacts → réponses → RDV → conversions, depuis `kpi_setting`) et des *indicateurs marketing* (vues et leads par plateforme, depuis `kpi_marketing`). Les graphiques sont réalisés avec Recharts (voir les pages « KPI commerciaux » et « KPI marketing » en figures 3.1c et 3.1d).

3.6 Intégration à la base Supabase

3.6.1 Principe et politique de sécurité

Chaque page interroge directement les vraies tables Supabase via des *hooks* dédiés (`useMissions`, `useKPICommercial`, `useKPIMarketing`, `useVehiclePositions`, `useAlertes`). Les tables étant protégées par RLS pour le seul rôle `authenticated`, le tableau de bord se connecte avec un compte de service dédié (solution temporaire en attendant l'authentification par utilisateur de Mounib).

3.6.2 Un blocage diagnostiqué puis résolu

Lors des premiers tests, la connexion échouait avec une erreur de permission : les requêtes partaient en tant qu'anonyme, or la RLS n'autorise que le rôle authentifié. En interrogeant **directement l'API REST de Supabase** (indépendamment de l'application), j'ai confirmé qu'il ne s'agissait pas d'un bug de mon code mais d'un besoin d'authentification. Une fois le compte de service fourni par Mounib configuré, la connexion aboutit et la RLS accorde bien l'accès en lecture. La vérification finale, menée par une requête de comptage sur chaque table, donne :

- `kpi_marketing` — **159 lignes réelles** lues avec succès ;
- `missions`, `adresses`, `kpi_setting`, `positions_vehicules` — tables accessibles mais **encore vides** (en attente de la migration Excel de Mounib et des positions GPS d'Israa).

Le « vide » constaté à l'écran ne provient donc pas du tableau de bord, mais du fait que ces tables ne sont pas encore alimentées. Un test complémentaire (insertion d'une ligne, lecture via la session du dashboard, puis suppression) a confirmé que la chaîne complète fonctionne de bout en bout.

3.7 Intégration avec le socle du CRM

Mon travail a été intégré par-dessus le socle applicatif d'Aurélien (tâches C1 et C2, déjà fusionnées). Cette intégration a demandé de résoudre des collisions de fichiers : le layout partagé `/admin/layout.tsx` d'Aurélien est conservé (il porte la navigation de tout l'espace interne), tandis que la « chrome » du dashboard (barre latérale, garde de connexion) a été déplacée dans un layout imbriqué propre à `/admin/dashboard`. Le travail est isolé sur une branche Git dédiée, destinée à une *Pull Request* conformément aux règles du dépôt (fusion soumise à relecture).

3.8 Refonte du design

Une première version, très animée (effets de survol, dégradés), a été retravaillée vers un style plus sobre : suppression des animations, cartes à bordure fine, palette neutre et usage ponctuel des couleurs de l'entreprise (bleu #0063A4, or #D3AF37).

3.9 Démarche de vérification

Chaque évolution a été vérifiée de façon complémentaire :

- vérification des types avec le compilateur TypeScript (`tsc -noEmit`) ;
- **build de production complet** (`next build`) après l'intégration du code de l'équipe, pour confirmer que l'ensemble compile (toutes les routes CRM et dashboard) ;
- test de l'appel réel à `OpenRouteService` avec la vraie clé ;
- test direct de l'API REST Supabase (connexion, RLS, comptage, insertion/lecture/suppression) pour valider la chaîne de données indépendamment de l'interface.

3.10 Difficultés rencontrées

3.10.1 Un cache de compilation corrompu

Le serveur local a cessé de démarrer (saturation mémoire, puis erreur interne de Turbopack). La cause n'était pas le code mais un cache `.next` corrompu ; sa suppression a résolu le problème — une illustration de l'importance de lire les journaux avant de conclure à un bug.

3.10.2 Des droits de base non ouverts au départ

Décrit à la section 3.6 : plutôt que de chercher une solution côté tableau de bord, la démarche a été de diagnostiquer précisément la cause (test direct de l'API) puis de formuler une demande claire à la bonne personne.

3.10.3 Dépendances inter-équipes

Une grande partie du lot L5 dépend de livrables d'autres stagiaires (API de Mounib, GPS d'Israa, KPI de David). L'enseignement principal a été de concevoir le tableau de bord pour qu'il reste **toujours démontrable** indépendamment de l'état des autres lots (repli polling, tolérance aux tables vides).

Chapitre 4

Conclusion et prochaines étapes

4.1 État d'avancement au 11 août 2026

Phase (cahier des charges)	État	Détail
A – Socle & maquettes	Terminée	4 pages, layout, navigation, POC carte
B – Cartographie & itinéraires	Terminée	Carte, géocodage, itinéraires ORS + ETA, gestion des erreurs d'API
C – Temps réel	Terminée	Abonnement Realtime + repli polling + badge de fraîcheur ; alertes lead chaud / mission en retard
D – KPI métier	En cours	Entonnoir et marketing connectés à Supabase ; reste les filtres de période ; données réelles partielles (<code>kpi_marketing</code> alimenté)
E – Recette & finitions	À venir	Prévue en fin de stage (validation croisée des KPI, optimisation, démonstration)

TABLE 4.1 – Avancement du lot L5

4.2 Prochaines étapes

- Filtres et sélection de période sur les KPI (comparaison N-1).
- Branchement des positions GPS réelles d'Israa (test croisé) et des données migrées par Mounib (`missions`, `kpi_setting`).
- Bascule du compte de service vers l'authentification par utilisateur (SSR) de Mounib.
- Exécution des scénarios de recette et validation croisée des chiffres KPI.
- Démonstration finale au gérant et documentation de passation ; ouverture de la Pull Request pour l'intégration.

Conclusion

[A COMPLETER : Conclusion personnelle à rédiger en fin de stage : bilan global par rapport aux objectifs, résultat de la démonstration finale, apport personnel du stage, suite envisagée.]

Annexe A

DD&RS — Développement durable et responsabilité sociétale

Le projet contribue à plusieurs objectifs de responsabilité :

- **Réduction du papier et de la double saisie** : le tableau de bord et le CRM remplacent des classeurs Excel manuels, limitant les erreurs et les impressions.
- **Optimisation des trajets** : le calcul d'itinéraires (ORS) et le suivi temps réel aident à limiter les kilomètres à vide, donc la consommation de carburant et les émissions.
- **Outils libres** : l'usage d'OpenStreetMap et de logiciels open source réduit la dépendance à des services propriétaires.

[A COMPLETER : Enrichir selon la grille DD&RS de l'ENSIIE (volets social, économique, environnemental).]

Annexe B

Structure de l'entreprise

[A COMPLETER : Insérer l'organigramme de RT MOVE et le décrire (référence en section 1.1).]

Annexe C

Détails techniques

C.1 Hook de suivi temps réel avec repli polling

```
1 const channel = supabase
2   .channel("positions-changes")
3   .on("postgres_changes",
4     { event: "*", schema: "public", table: "positions_vehicules" },
5     () => fetchVehiclePositions())
6   .subscribe((status) => {
7     if (status === "SUBSCRIBED") {
8       setConnectionStatus("realtime"); stopPolling();
9     } else if (["CHANNEL_ERROR", "TIMED_OUT", "CLOSED"].includes(status)) {
10      setConnectionStatus("polling"); startPolling();
11    }
12  });
13 // Repli si le canal n'atteint jamais SUBSCRIBED
14 fallbackTimer = setTimeout(() => { startPolling(); }, 8000);
```

Listing C.1 – Extrait simplifié – lib/hooks/use-supabase.ts

C.2 Alertes de pilotage

```
1 const today = new Date().toISOString().split("T")[0];
2 // Leads chauds : prospects crees aujourd'hui
3 const { data: prospects } = await supabase
4   .from("prospects").select("id, nom, prenom, type, source, date_creation")
5   .eq("date_creation", today).order("date_creation", { ascending: false });
6 // Missions en retard : date passee, statut non cloture
7 const { data: missions } = await supabase
8   .from("missions").select("id, date_operation, statut")
9   .lt("date_operation", today)
10  .in("statut", ["a_planifier", "planifiee", "en_cours"]);
```

Listing C.2 – Extrait simplifié – lib/hooks/use-alertes.ts

C.3 Appel sécurisé à OpenRouteService (côté serveur)

```
1 const key = process.env.ORS_API_KEY; // cle jamais exposee au client
2 const r = await fetch(ORS_URL, { method: "POST", headers, body });
3 if (!r.ok) {
4   return NextResponse.json({ error: `ORS ${r.status}` },
5     { status: r.status === 429 ? 429 : 502 }); // quota vs indisponible
```

6 }

Listing C.3 – Extrait simplifié – app/api/directions/route.ts

Bibliographie

- [1] Documentation Next.js. <https://nextjs.org/docs>
- [2] Documentation React. <https://react.dev>
- [3] Documentation Tailwind CSS. <https://tailwindcss.com/docs>
- [4] shadcn/ui. <https://ui.shadcn.com>
- [5] Leaflet.js. <https://leafletjs.com>
- [6] OpenStreetMap. <https://www.openstreetmap.org>
- [7] OpenRouteService. <https://openrouteservice.org>
- [8] Recharts. <https://recharts.org>
- [9] Documentation Supabase (client JS, Realtime, RLS). <https://supabase.com/docs>